

编译原理

第十一讲：代码优化

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 5 月 14 日



福州大学
FUZHOU UNIVERSITY

目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



同一段代码，不同的性能

原始写法 ($O(n^2)$)

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        sum += a[i];
```

优化写法 ($O(n)$)

```
for (int i = 0; i < n; i++)  
    sum += a[i] * n;
```

同一段代码，不同的性能

原始写法 ($O(n^2)$)

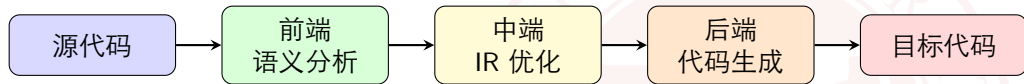
```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        sum += a[i];
```

优化写法 ($O(n)$)

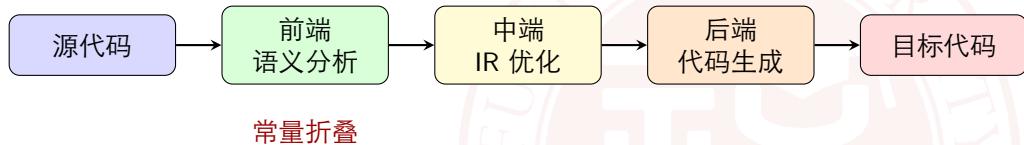
```
for (int i = 0; i < n; i++)  
    sum += a[i] * n;
```

- 编译器的核心使命之一：在保持语义等价的前提下，提升程序运行效率。
- 本讲将介绍从“中端 IR”到“目标代码”阶段常见的优化技术。

优化的层次与编译器架构



优化的层次与编译器架构



- 前端优化：基于 AST，进行局部、简单的变换（如常量折叠）。

优化的层次与编译器架构



- 前端优化：基于 AST，进行局部、简单的变换（如常量折叠）。
- 中端优化：基于 IR（如 LLVM IR、三地址码），进行全局、过程内/间分析。

优化的层次与编译器架构



- 前端优化：基于 AST，进行局部、简单的变换（如常量折叠）。
- 中端优化：基于 IR（如 LLVM IR、三地址码），进行全局、过程内/间分析。
- 后端优化：面向具体目标机器，考虑寄存器、流水线、指令集特性。

常量折叠 (Constant Folding)

- 阶段：语义分析 / 前端
- 核心思想：在 AST 遍历过程中，若子节点皆为常量，直接计算父节点值。

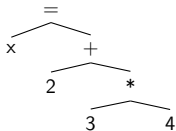


常量折叠 (Constant Folding)

- 阶段：语义分析 / 前端
- 核心思想：在 AST 遍历过程中，若子节点皆为常量，直接计算父节点值。

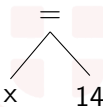
优化前 (AST)

```
int x = 2 + 3 * 4;
```



优化后 (AST)

```
int x = 14;
```

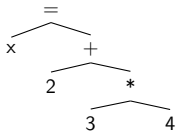


常量折叠 (Constant Folding)

- 阶段：语义分析 / 前端
- 核心思想：在 AST 遍历过程中，若子节点皆为常量，直接计算父节点值。

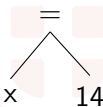
优化前 (AST)

```
int x = 2 + 3 * 4;
```



优化后 (AST)

```
int x = 14;
```



算法

对 AST 做后序遍历：若当前节点操作符的所有子节点均为字面常量，则直接用计算结果替换该子树。

常量传播 (Constant Propagation)

- 阶段：中端 IR
- 核心思想：维护一个 SymbolTable，记录 `var` $\rightarrow$ `constant` 的映射，并替换后续引用。



常量传播 (Constant Propagation)

- 阶段：中端 IR
- 核心思想：维护一个 SymbolTable，记录 `var` $\rightarrow$ `constant` 的映射，并替换后续引用。

优化前三地址码

```
a = 5
b = a + 1
c = b * 2
```

优化后三地址码

```
a = 5
b = 6      ; a->5
c = 12     ; b->6
```

常量传播 (Constant Propagation)

- 阶段：中端 IR
- 核心思想：维护一个 SymbolTable，记录 $\text{var} \rightarrow \text{constant}$ 的映射，并替换后续引用。

优化前三地址码

```
a = 5
b = a + 1
c = b * 2
```

优化后三地址码

```
a = 5
b = 6      ; a->5
c = 12     ; b->6
```

与常量折叠的联系

传播常和折叠交替执行：传播后暴露新的折叠机会，折叠后又可继续传播。

代数简化 (Algebraic Simplification)

- 阶段：中端 IR
- 核心思想：通过模式匹配，将昂贵或冗余运算替换为等价但更高效的形式。



代数简化 (Algebraic Simplification)

- 阶段：中端 IR
- 核心思想：通过模式匹配，将昂贵或冗余运算替换为等价但更高效的形式。

模式	替换为	说明
$x * 0$	0	乘法零元
$x * 1$	x	乘法单位元
$x * 2$	$x + x$ 或移位指令	强度削弱
$(a \ll 3) \ll 2$	$a \ll 5$	移位合并

代数简化 (Algebraic Simplification)

- 阶段：中端 IR
- 核心思想：通过模式匹配，将昂贵或冗余运算替换为等价但更高效的形式。

模式	替换为	说明
$x * 0$	0	乘法零元
$x * 1$	x	乘法单位元
$x * 2$	$x + x$ 或移位指令	强度削弱
$(a \ll 3) \ll 2$	$a \ll 5$	移位合并

算法

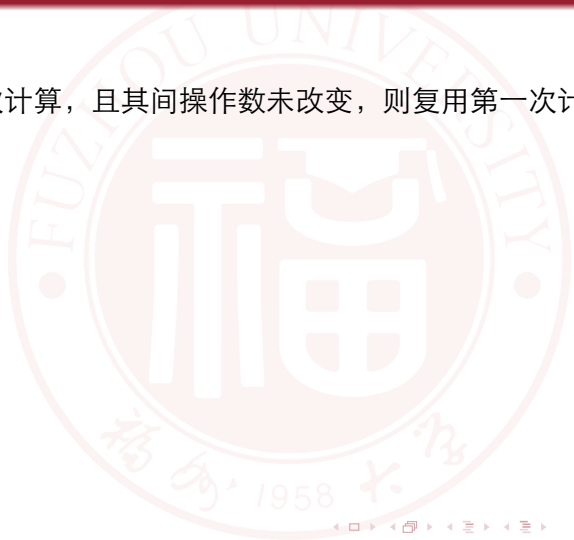
遍历 IR 指令，对每条指令匹配左侧模式；若匹配成功，则替换为右侧等价表达式。

常见运算的代价与效率

运算类型	延迟 (Cycles)	吞吐量 (IPC)	编译器视角
整数加/减法	1	4 - 6	极其廉价
整数乘法 (32/64 位)	3 - 5	1 - 2	比较快
浮点加法 (FADD)	3 - 4	2	延迟显著
浮点乘法 (FMUL)	3 - 5	2	
整数/浮点除法	10 - 50+	< 0.5	昂贵

公共子表达式消除（CSE）

- 阶段：中端 IR
- 核心思想：若同一表达式被多次计算，且其间操作数未改变，则复用第一次计算的结果。



公共子表达式消除 (CSE)

- 阶段：中端 IR
- 核心思想：若同一表达式被多次计算，且其间操作数未改变，则复用第一次计算的结果。

优化前三地址码

```
t1 = a + b
t2 = a + b    ; 重复!
t3 = t1 * 2
t4 = t2 * 2    ; 重复!
```

优化后三地址码

```
t1 = a + b
t2 = t1        ; 复用
t3 = t1 * 2
t4 = t3        ; 复用
```

公共子表达式消除 (CSE)

- 阶段：中端 IR
- 核心思想：若同一表达式被多次计算，且其间操作数未改变，则复用第一次计算的结果。

优化前三地址码

```
t1 = a + b
t2 = a + b    ; 重复!
t3 = t1 * 2
t4 = t2 * 2    ; 重复!
```

优化后三地址码

```
t1 = a + b
t2 = t1        ; 复用
t3 = t1 * 2
t4 = t3        ; 复用
```

算法：值编号 (Value Numbering)

为每个表达式计算一个 Hash 值 (值编号)；若两条指令的值编号相同，则它们是等价的，可以消除冗余。

实战：观察 LLVM 的 -O1 优化

常数折叠与传播

```
1 int foo(int x) {  
2     int a = 2 + 3;  
3     int b = a * 1;  
4     int c = x * 0;  
5     int d = x << 1;  
6     return b + c + d;  
7 }
```

CSE

```
1 int bar(int x, int y) {  
2     int a = x + y;  
3     int b = x + y;  
4     int c = a * 2;  
5     int d = b * 2;  
6     return c + d;  
7 }
```

实战：观察 LLVM 的 -O1 优化

常数折叠与传播

```
1 int foo(int x) {  
2     int a = 2 + 3;  
3     int b = a * 1;  
4     int c = x * 0;  
5     int d = x << 1;  
6     return b + c + d;  
7 }
```

```
; -O2 后  
%2 = shl i32 %0, 1  
%3 = add nsw i32 %2, 5  
ret i32 %3
```

CSE

```
1 int bar(int x, int y) {  
2     int a = x + y;  
3     int b = x + y;  
4     int c = a * 2;  
5     int d = b * 2;  
6     return c + d;  
7 }
```

```
; -O2 后  
%3 = add nsw i32 %1, %0  
%4 = shl nsw i32 %3, 2  
ret i32 %4
```

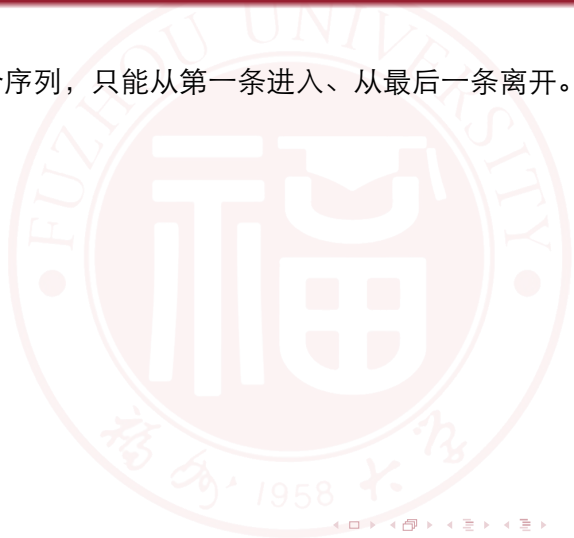
目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



划分基本块 (Basic Blocks)

- 阶段：IR 后处理
- 定义：基本块是顺序执行的指令序列，只能从第一条进入、从最后一条离开。



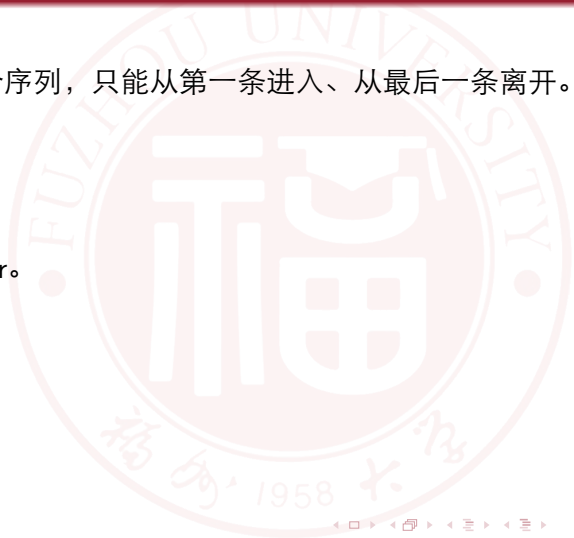
划分基本块 (Basic Blocks)

- 阶段：IR 后处理
- 定义：基本块是顺序执行的指令序列，只能从第一条进入、从最后一条离开。

Leader 识别法

- ① 第一条指令是 Leader。
- ② 任何跳转指令的目标是 Leader。
- ③ 任何跳转指令的下一条是 Leader。

Leader 之间的指令构成一个基本块。



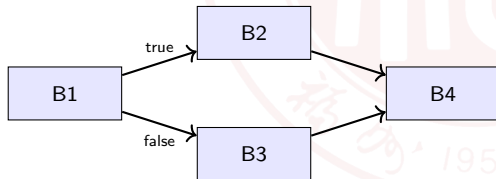
划分基本块 (Basic Blocks)

- 阶段：IR 后处理
- 定义：基本块是顺序执行的指令序列，只能从第一条进入、从最后一条离开。

Leader 识别法

- ① 第一条指令是 Leader。
- ② 任何跳转指令的目标是 Leader。
- ③ 任何跳转指令的下一条是 Leader。

Leader 之间的指令构成一个基本块。



构建控制流图（CFG）

- 阶段：IR 后处理
- 核心思想：在基本块之间连边，形成有向图，刻画程序的执行路径。

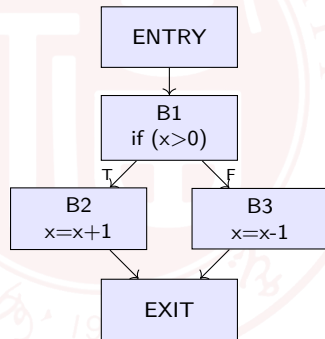


构建控制流图 (CFG)

- 阶段：IR 后处理
- 核心思想：在基本块之间连边，形成有向图，刻画程序的执行路径。

连边规则

- ① 若 BB 结尾是条件/无条件跳转，则连向目标 BB。
- ② 若 BB 结尾不是跳转 (fall-through)，则连向物理上的下一块。
- ③ 返回指令没有出边。



活跃变量分析 (Live Variable Analysis)

- 阶段：中端 IR
- 直觉：一个变量在某点活跃，意味着它当前持有的值在之后还会被读取。
- 类比：检查“过期的食物”——后面再也没被用到的变量就是“过期”的。

活跃变量分析 (Live Variable Analysis)

- 阶段：中端 IR
- 直觉：一个变量在某点活跃，意味着它当前持有的值在之后还会被读取。
- 类比：检查“过期的食物”——后面再也没被用到的变量就是“过期”的。

数据流方程

$$\text{LiveIn}(B) = \text{Use}(B) \cup (\text{LiveOut}(B) - \text{Def}(B))$$

$$\text{LiveOut}(B) = \bigcup_{S \in \text{succ}(B)} \text{LiveIn}(S)$$

算法

- **Use**：在赋值前被读取的变量集合。
- **Def**：被赋值的变量集合。
- 计算顺序：倒序遍历基本块，从出口向入口迭代至不动点。

死代码消除 (Dead Code Elimination, DCE)

- 阶段：中端 IR
- 核心思想：若一个赋值指令的目标变量在当前点“不活跃”，则该指令可直接删除。



死代码消除 (Dead Code Elimination, DCE)

- 阶段：中端 IR
- 核心思想：若一个赋值指令的目标变量在当前点“不活跃”，则该指令可直接删除。

优化前

```
1 int deadcode(int x) {  
2     int a = x + 10;    // dead  
3     int b = x * 2;     // live  
4     return b; }
```

```
; -00  
%6 = add nsw i32 %5, 10  
store i32 %6, ptr %3  
%8 = mul nsw i32 %7, 2  
store i32 %8, ptr %4  
%9 = load i32, ptr %4  
ret i32 %9
```

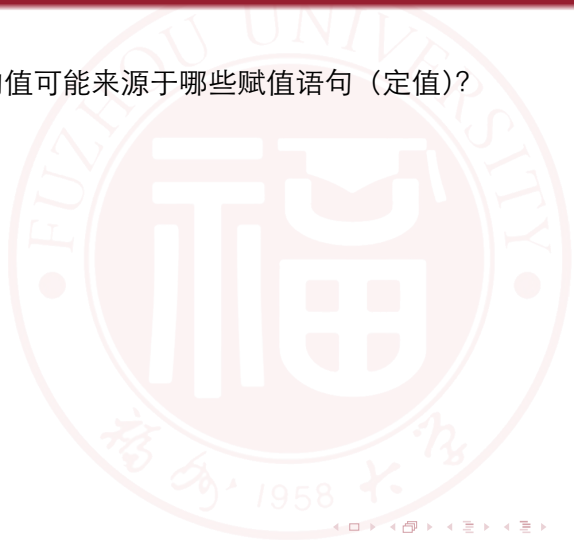
优化后

```
1 int deadcode(int x) {  
2     int b = x * 2;  
3     return b;  
4 }
```

```
; -02  
%2 = shl nsw i32 %0, 1  
ret i32 %2
```

到达定值分析 (Reaching Definitions)

- 阶段：中端 IR
- 直觉：在程序的某一点，变量的值可能来源于哪些赋值语句（定值）？



到达定值分析 (Reaching Definitions)

- 阶段：中端 IR
- 直觉：在程序的某一点，变量的值可能来源于哪些赋值语句（定值）？

数据流方程

$$\text{In}[B] = \bigcup_{P \in \text{pred}(B)} \text{Out}[P]$$

$$\text{Out}[B] = (\text{In}[B] - \text{Kill}[B]) \cup \text{Gen}[B]$$

到达定值分析 (Reaching Definitions)

- 阶段：中端 IR
- 直觉：在程序的某一点，变量的值可能来源于哪些赋值语句（定值）？

数据流方程

$$\begin{aligned} \text{In}[B] &= \bigcup_{P \in \text{pred}(B)} \text{Out}[P] \\ \text{Out}[B] &= (\text{In}[B] - \text{Kill}[B]) \cup \text{Gen}[B] \end{aligned}$$

算法

- **Gen**：基本块内产生的定值。
- **Kill**：基本块内杀死的（被覆盖的）定值。
- 应用：常量传播、拷贝传播、SSA 构造的前置步骤。

目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



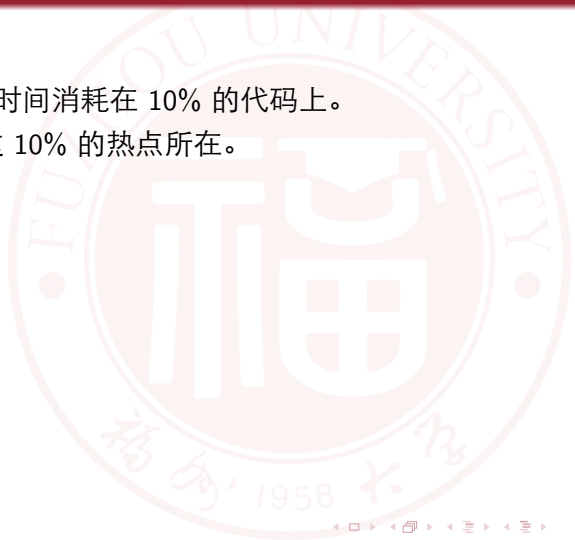
90/10 法则与循环的重要性

- **90/10 法则**：程序 90% 的运行时间消耗在 10% 的代码上。



90/10 法则与循环的重要性

- **90/10 法则**：程序 90% 的运行时间消耗在 10% 的代码上。
- 循环（尤其是嵌套循环）正是这 10% 的热点所在。



90/10 法则与循环的重要性

- **90/10 法则**：程序 90% 的运行时间消耗在 10% 的代码上。
- 循环（尤其是嵌套循环）正是这 10% 的热点所在。

核心观点

循环内部哪怕减少一条指令，由于被执行成千上万次，累积的性能提升也是巨大的。

90/10 法则与循环的重要性

- **90/10 法则**：程序 90% 的运行时间消耗在 10% 的代码上。
- 循环（尤其是嵌套循环）正是这 10% 的热点所在。

核心观点

循环内部哪怕减少一条指令，由于被执行成千上万次，累积的性能提升也是巨大的。



循环优化一般流程

循环不变式外提 (LICM)

- 阶段：中端 IR
- 核心思想：若运算的操作数都在循环外定值，则将其搬移至 Pre-header（循环入口处的前置块）。



循环不变式外提 (LICM)

- 阶段：中端 IR
- 核心思想：若运算的操作数都在循环外定值，则将其搬移至 Pre-header（循环入口处的前置块）。

优化前

```
1  for (int i = 0; i < n; i++) {  
2      int t = m * 10;  // m 不变  
3      sum += a[i] + t;  
4  }
```

循环不变式外提 (LICM)

- 阶段：中端 IR
- 核心思想：若运算的操作数都在循环外定值，则将其搬移至 Pre-header（循环入口处的前置块）。

优化前

```
1 for (int i = 0; i < n; i++) {  
2     int t = m * 10; // m 不变  
3     sum += a[i] + t;  
4 }
```

优化后

```
1 int t = m * 10; // 外提  
2 for (int i = 0; i < n; i++) {  
3     sum += a[i] + t;  
4 }
```

循环不变式外提 (LICM)

- 阶段：中端 IR
- 核心思想：若运算的操作数都在循环外定值，则将其搬移至 Pre-header（循环入口处的前置块）。

优化前

```
1 for (int i = 0; i < n; i++) {  
2     int t = m * 10; // m 不变  
3     sum += a[i] + t;  
4 }
```

优化后

```
1 int t = m * 10; // 外提  
2 for (int i = 0; i < n; i++) {  
3     sum += a[i] + t;  
4 }
```

```
; -O2 后, m*10 在循环外计算  
%6 = mul nsw i32 %2, 10      ; Pre-header  
...  
%16 = insertelement <4 x i32> poison, i32 %6, i64 0 ; 循环内复用
```

强度削弱 (Strength Reduction)

- 阶段：中端 IR
- 核心思想：将昂贵的运算（如乘法）替换为便宜的运算（如加法）。



强度削弱 (Strength Reduction)

- 阶段：中端 IR
- 核心思想：将昂贵的运算（如乘法）替换为便宜的运算（如加法）。

优化前

```
1 for (int i = 0; i < n; i++) {  
2     sum += i * 4;    // 乘法  
3 }
```

优化后

```
1 int tmp = 0;  
2 for (int i = 0; i < n; i++) {  
3     sum += tmp;  
4     tmp += 4;        // 加法  
5 }
```

强度削弱 (Strength Reduction)

- 阶段：中端 IR
- 核心思想：将昂贵的运算（如乘法）替换为便宜的运算（如加法）。

优化前

```
1 for (int i = 0; i < n; i++) {  
2     sum += i * 4;    // 乘法  
3 }
```

优化后

```
1 int tmp = 0;  
2 for (int i = 0; i < n; i++) {  
3     sum += tmp;  
4     tmp += 4;        // 加法  
5 }
```

算法

寻找归纳变量（如 i ），将 $i \times \text{const}$ 替换为每次迭代执行 $\text{temp} += \text{const}$ 。

循环展开 (Loop Unrolling)

- 阶段：中端 IR
- 核心思想：按因子复制循环体，减少循环控制开销，增加指令级并行性。



循环展开 (Loop Unrolling)

- 阶段：中端 IR
- 核心思想：按因子复制循环体，减少循环控制开销，增加指令级并行性。

优化前

```
1  for (int i = 0; i < 4; i++)  
2      sum += a[i];
```

优化后 (因子 4)

```
1  sum += a[0];  
2  sum += a[1];  
3  sum += a[2];  
4  sum += a[3];
```

循环展开 (Loop Unrolling)

- 阶段：中端 IR
- 核心思想：按因子复制循环体，减少循环控制开销，增加指令级并行性。

优化前

```
1 for (int i = 0; i < 4; i++)  
2     sum += a[i];
```

优化后 (因子 4)

```
1 sum += a[0];  
2 sum += a[1];  
3 sum += a[2];  
4 sum += a[3];
```

LLVM -O2 实际输出

对于动态边界 n ，LLVM 会生成：向量化主循环（每次处理 16 个元素）+ 收尾标量循环。

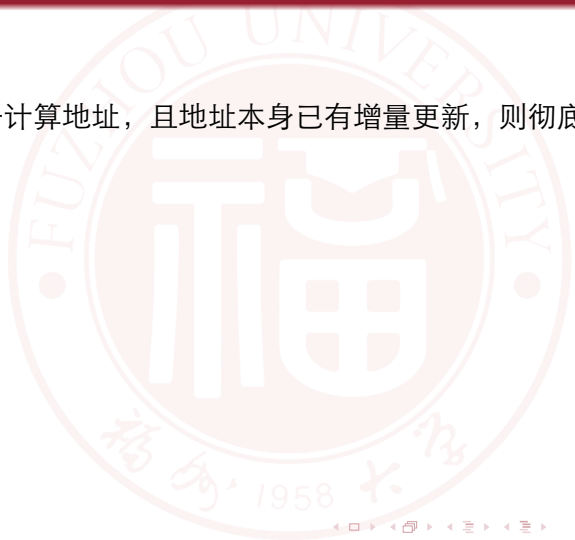
循环展开实战：LLVM 的向量化

```
1 ; sum_array -O2 (部分)
2 %14 = phi <4 x i32> [ zeroinitializer, %12 ], [ %28, %14 ]
3 %20 = getelementptr inbounds nuw i32, ptr %0, i64 %15
4 %24 = load <4 x i32>, ptr %20, align 4
5 %28 = add <4 x i32> %24, %16
```

- `<4 x i32>` 表示一次处理 4 个 32 位整数的 SIMD 向量。
- LLVM 自动将循环展开并转换为 NEON/AVX 向量指令。

归纳变量消除 (Induction Variable Elimination)

- 阶段：中端 IR
- 核心思想：若循环索引 i 只用于计算地址，且地址本身已有增量更新，则彻底删除 i 。



归纳变量消除 (Induction Variable Elimination)

- 阶段：中端 IR
- 核心思想：若循环索引 i 只用于计算地址，且地址本身已有增量更新，则彻底删除 i 。

优化前

```
1  for (int i = 0; i < n; i++)  
2      a[i] = i;
```

优化后

```
1  int *p = a;  
2  for (; p < a + n; p++)  
3      *p = p - a;
```

归纳变量消除 (Induction Variable Elimination)

- 阶段：中端 IR
- 核心思想：若循环索引 i 只用于计算地址，且地址本身已有增量更新，则彻底删除 i 。

优化前

```
1 for (int i = 0; i < n; i++)  
2     a[i] = i;
```

优化后

```
1 int *p = a;  
2 for (; p < a + n; p++)  
3     *p = p - a;
```

更进一步

若数组访问被替换为指针的递增 ($p += 4$)，则索引变量 i 可被完全消除。

目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



函数内联 (Function Inlining)

- 阶段：中端 / 前端
- 核心思想：用函数体替换调用点，消除调用开销（栈帧建立、参数传递、跳转返回）。



函数内联 (Function Inlining)

- 阶段：中端 / 前端
- 核心思想：用函数体替换调用点，消除调用开销（栈帧建立、参数传递、跳转返回）。

优化前

```
1 static int add(int a, int b) {  
2     return a + b; }  
3 int test(int x, int y) {  
4     return add(x, y) * 2; }
```

优化后

```
1 int test(int x, int y) {  
2     return (x + y) * 2;  
3 }
```

函数内联 (Function Inlining)

- 阶段：中端 / 前端
- 核心思想：用函数体替换调用点，消除调用开销（栈帧建立、参数传递、跳转返回）。

优化前

```
1 static int add(int a, int b) {  
2     return a + b; }  
3 int test(int x, int y) {  
4     return add(x, y) * 2; }
```

```
; -00: 显式 call  
%7 = call i32 @add(...)  
%8 = mul nsw i32 %7, 2
```

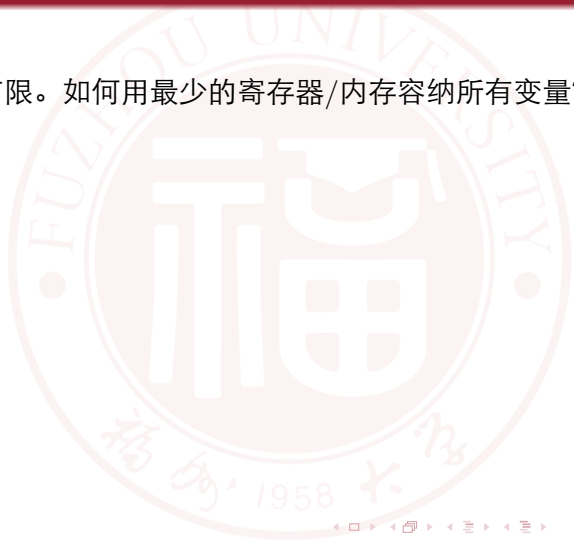
优化后

```
1 int test(int x, int y) {  
2     return (x + y) * 2;  
3 }
```

```
; -02: add 被内联  
%3 = add nsw i32 %1, %0  
%4 = shl nsw i32 %3, 1
```

寄存器分配 (Register Allocation)

- 阶段：目标代码生成
- 核心问题：变量无限，寄存器有限。如何用最少的寄存器/内存容纳所有变量？



寄存器分配 (Register Allocation)

- 阶段：目标代码生成
- 核心问题：变量无限，寄存器有限。如何用最少的寄存器/内存容纳所有变量？

图着色算法 (Graph Coloring)

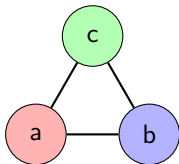
- ① 构建冲突图：若两个变量同时活跃，则它们之间连一条边。
- ② 用 K 种颜色 ($K =$ 寄存器个数) 给图着色。
- ③ 若颜色不够，将某个变量“溢出” (Spilling) 到内存栈帧。

寄存器分配 (Register Allocation)

- 阶段：目标代码生成
- 核心问题：变量无限，寄存器有限。如何用最少的寄存器/内存容纳所有变量？

图着色算法 (Graph Coloring)

- ① 构建冲突图：若两个变量同时活跃，则它们之间连一条边。
- ② 用 K 种颜色 ($K =$ 寄存器个数) 给图着色。
- ③ 若颜色不够，将某个变量“溢出” (Spilling) 到内存栈帧。



a, b, c 两两冲突
至少需要 3 个寄存器

溢出计算：谁该被踢出？

- 当 K 种颜色（寄存器数量）无法满足冲突图时，需要执行 溢出（Spilling）。
- 目标：最小化内存访问指令，根据“溢出代价（Spill Cost）”排队。



溢出计算：谁该被踢出？

- 当 K 种颜色（寄存器数量）无法满足冲突图时，需要执行 **溢出（Spilling）**。
- 目标：最小化内存访问指令，根据“溢出代价（Spill Cost）”排队。

核心评估公式：溢出权重

$$W(v) = \frac{\sum_{p \in \text{uses}(v) \cup \text{defs}(v)} 10^{\text{depth}(p)}}{\text{degree}(v)}$$

- 访问频率：统计变量被读取（use）和赋值（def）的总次数。
- 循环嵌套深度（ 10^{depth} ）：循环内访问权重大幅提升，确保“高频变量”稳坐寄存器。
- 冲突度（ $\text{degree}(v)$ ）：溢出“社交名媛”可一次性释放大量约束。

溢出优先级的“黑名单”

溢出优先级	变量特征	理由
第一级（最先溢出）	长距离存活且低频使用	定义后过了很久才被使用，中间占着坑位却不干活。
第二级	冲突度极高的变量	它一个人占了太多人的“颜色”，把它移走能盘活一盘棋。
第三级	非循环体内的变量	在非热点代码区，偶尔进出一次内存的开销可以接受。
禁止溢出	刚刚被载入或即将被使用的变量	刚从内存读出来又立刻踢回去会造成死循环（Spill Code 污染）。

溢出优先级的“黑名单”

溢出优先级	变量特征	理由
第一级（最先溢出）	长距离存活且低频使用	定义后过了很久才被使用，中间占着坑位却不干活。
第二级	冲突度极高的变量	它一个人占了太多人的“颜色”，把它移走能盘活一盘棋。
第三级	非循环体内的变量	在非热点代码区，偶尔进出一次内存的开销可以接受。
禁止溢出	刚刚被载入或即将被使用的变量	刚从内存读出来又立刻踢回去会造成死循环（Spill Code 污染）。

核心准则

保循环、踢闲人、救大众（高冲突度）

溢出过程中的“省钱”秘籍

- 即便决定了溢出谁，编译器还会进一步减少指令开销：



溢出过程中的“省钱”秘籍

- 即便决定了溢出谁，编译器还会进一步减少指令开销：

重新物质化（Rematerialization）

若变量的值可通过简单指令（如 $x = 5$ 或 $x = \text{sp} + \text{offset}$ ）重新算出，不执行 **Store**，需要时直接重新计算。

溢出过程中的“省钱”秘籍

- 即便决定了溢出谁，编译器还会进一步减少指令开销：

重新物质化（Rematerialization）

若变量的值可通过简单指令（如 $x = 5$ 或 $x = sp + \text{offset}$ ）重新算出，不执行 **Store**，需要时直接重新计算。

拆分存活区间（Live Range Splitting）

若变量只在某一小段很忙、其他地方很闲，将其拆成两段：闲段住内存，忙段继续抢寄存器。

溢出过程中的“省钱”秘籍

- 即便决定了溢出谁，编译器还会进一步减少指令开销：

重新物质化 (Rematerialization)

若变量的值可通过简单指令（如 $x = 5$ 或 $x = sp + \text{offset}$ ）重新算出，不执行 **Store**，需要时直接重新计算。

拆分存活区间 (Live Range Splitting)

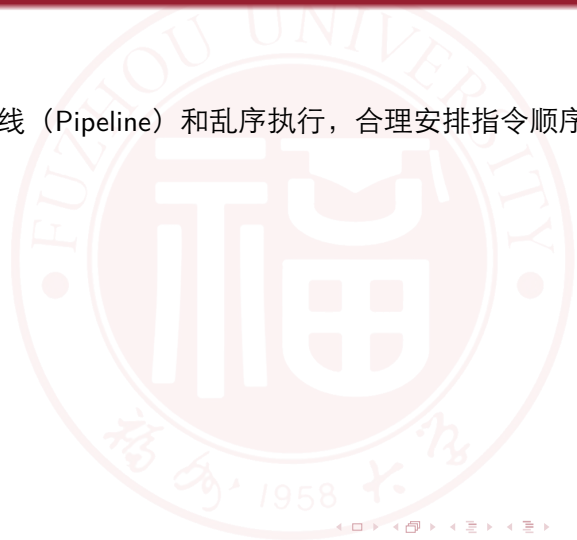
若变量只在某一小段很忙、其他地方很闲，将其拆成两段：闲段住内存，忙段继续抢寄存器。

总结

寄存器分配的溢出策略 = 语义保持 + 成本模型 + 启发式选择。优先踢掉深层循环外、访问次数少、且挡了别人路的变量。

指令调度 (Instruction Scheduling)

- 阶段：目标代码生成
- 核心思想：现代 CPU 采用流水线 (Pipeline) 和乱序执行，合理安排指令顺序可减少停顿 (Stall)。



指令调度 (Instruction Scheduling)

- 阶段：目标代码生成
- 核心思想：现代 CPU 采用流水线（Pipeline）和乱序执行，合理安排指令顺序可减少停顿（Stall）。

列表调度 (List Scheduling)

- ① 根据数据依赖生成 DAG（有向无环图）。
- ② 按优先级（如关键路径长度）选择就绪指令填入时间槽。
- ③ 目标：最大化指令级并行（ILP），减少流水线气泡。

指令调度 (Instruction Scheduling)

- 阶段：目标代码生成
- 核心思想：现代 CPU 采用流水线 (Pipeline) 和乱序执行，合理安排指令顺序可减少停顿 (Stall)。

列表调度 (List Scheduling)

- ① 根据数据依赖生成 DAG (有向无环图)。
- ② 按优先级 (如关键路径长度) 选择就绪指令填入时间槽。
- ③ 目标：最大化指令级并行 (ILP)，减少流水线气泡。

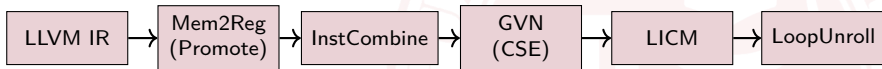
周期	朴素顺序	调度后
1	LOAD a	LOAD a
2	ADD a, b	LOAD b
3	(stall: 等待 a)	ADD a, b
4	LOAD b	STORE c
5	ADD a, b	...

现代编译器的 Pass 机制

- 在 LLVM 等现代编译器中，每个优化被实现为一个独立的 **Pass**（变换函数）。
- Pass 以流水线方式依次作用于 IR，每个 Pass 只做一件事。

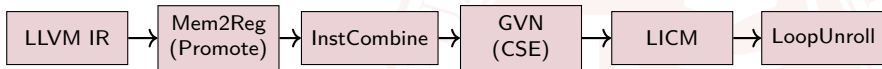
现代编译器的 Pass 机制

- 在 LLVM 等现代编译器中，每个优化被实现为一个独立的 **Pass**（变换函数）。
- Pass 以流水线方式依次作用于 IR，每个 Pass 只做一件事。



现代编译器的 Pass 机制

- 在 LLVM 等现代编译器中，每个优化被实现为一个独立的 **Pass**（变换函数）。
- Pass 以流水线方式依次作用于 IR，每个 Pass 只做一件事。



- **Mem2Reg**: 将内存中的局部变量提升到 SSA 寄存器。
- **InstCombine**: 指令组合与代数简化。
- **GVN**: 全局值编号，实现跨基本块的 CSE。
- **LICM** / **LoopUnroll**: 循环不变式外提与循环展开。

总结：优化的权衡艺术

- 代码优化不是“免费的午餐”，需要在多个维度间权衡：



总结：优化的权衡艺术

- 代码优化不是“免费的午餐”，需要在多个维度间权衡：

维度	选项 A	选项 B
编译时间	快速编译 (-O0)	深度优化 (-O3)
运行时间	解释执行	JIT 编译
代码体积	循环展开 (大)	保守策略 (小)
调试友好	保留变量 (-O0)	激进优化 (-O2)
能耗	低功耗模式	高性能模式

总结：优化的权衡艺术

- 代码优化不是“免费的午餐”，需要在多个维度间权衡：

维度	选项 A	选项 B
编译时间	快速编译 (-O0)	深度优化 (-O3)
运行时间	解释执行	JIT 编译
代码体积	循环展开 (大)	保守策略 (小)
调试友好	保留变量 (-O0)	激进优化 (-O2)
能耗	低功耗模式	高性能模式

核心

编译优化 = 语义保持 + 成本模型 + 启发式搜索

目录

- ① 基础优化——从“能跑”到“跑得快”
- ② 数据流分析——代码的“交通地图”
- ③ 循环优化——性能的杠杆
- ④ 后端优化——通往机器的最后一公里
- ⑤ 小结与作业



本讲小结

- 基础优化：常量折叠、常量传播、代数简化、CSE 是编译器最常用、开销最低的局部优化。



本讲小结

- 基础优化：常量折叠、常量传播、代数简化、CSE 是编译器最常用、开销最低的局部优化。
- 数据流分析：CFG、活跃变量分析、到达定值为全局优化提供了精确的程序状态信息。

本讲小结

- 基础优化：常量折叠、常量传播、代数简化、CSE 是编译器最常用、开销最低的局部优化。
- 数据流分析：CFG、活跃变量分析、到达定值为全局优化提供了精确的程序状态信息。
- 循环优化：LICM、强度削弱、循环展开、归纳变量消除是提升性能的核心杠杆。

本讲小结

- 基础优化：常量折叠、常量传播、代数简化、CSE 是编译器最常用、开销最低的局部优化。
- 数据流分析：CFG、活跃变量分析、到达定值为全局优化提供了精确的程序状态信息。
- 循环优化：LICM、强度削弱、循环展开、归纳变量消除是提升性能的核心杠杆。
- 后端技术：函数内联、图着色寄存器分配、列表调度指令将 IR 高效映射到物理机器。

本讲小结

- 基础优化：常量折叠、常量传播、代数简化、CSE 是编译器最常用、开销最低的局部优化。
- 数据流分析：CFG、活跃变量分析、到达定值为全局优化提供了精确的程序状态信息。
- 循环优化：LICM、强度削弱、循环展开、归纳变量消除是提升性能的核心杠杆。
- 后端技术：函数内联、图着色寄存器分配、列表调度指令将 IR 高效映射到物理机器。
- **Pass 机制**：现代编译器将优化拆解为独立 Pass，按流水线方式组合，便于扩展与复用。

作业 I

① 活跃变量分析手工推导

给定如下三地址码片段，划分基本块并构建 CFG，然后手工计算每个基本块的 LiveIn 与 LiveOut：

```
B1:  a = 1
      b = 2
      if (a < b) goto B3
B2:  c = a + b
      goto B4
B3:  c = a - b
B4:  d = c + 1
      return d
```

作业 II

② LLVM IR 观察

写一段包含多重循环、多维数组访问的 C 代码，分别用 `clang -O0 -S -emit-llvm` 和 `clang -O2 -S -emit-llvm` 生成 LLVM IR，对比分析编译器进行了哪些优化（至少指出 3 种）。